



UNS
UNIVERSIDAD
NACIONAL DEL SANTA

CURSO DE APLICACIONES MÓVILES

Desarrolla. Conecta. Innova.
Lleva tus ideas al siguiente nivel.



Flutter



Kotlin



React Native



Android



iOS

```
import 'package:flutter  
class HomePage extends  
@override  
Widget build(BuildContext  
return Scaffold(  
  appBar: AppBar(  
    body: Center(  
      child: Text(  
        'Hola, Mundo!!  
    )  
  )  
)  
)
```



**DESARROLLO
MULTIPLATAFORMA**
Crea apps para Android
e iOS con un solo código.



**INTERFACES
MODERNAS**
Diseña experiencias
atractivas y funcionales.



**CONECTIVIDAD
Y SERVICIOS**
Integra APIs, bases de datos
y servicios en la nube.



**SEGURIDAD
Y RENDIMIENTO**
Desarrolla aplicaciones
seguras, rápidas y
optimizadas.



**PUBLICACIÓN
Y DESPLIEGUE**
Publica tu app en Google
Play y App Store.



DESARROLLO
DE APPS



UI/UX
MÓVIL



APIS Y
SERVICIOS



BASES DE
DATOS



PRUEBAS Y
DEPLOY



Docente:

Dr. Mirko Manrique Ronceros

INTRODUCCIÓN

En el desarrollo de aplicaciones móviles Android, comprender la estructura y los elementos que conforman una aplicación resulta esencial para construir soluciones funcionales, escalables y orientadas al usuario. Los componentes de una aplicación Android definen la arquitectura básica sobre la cual se construyen las funcionalidades. Entre estos componentes fundamentales se encuentran las Activities, que representan pantallas de interacción; los Services, encargados de operaciones en segundo plano; los Broadcast Receivers, que permiten la recepción de eventos del sistema o de otras aplicaciones; y los Content Providers, que facilitan el acceso y la gestión de datos compartidos entre aplicaciones. Cada uno de estos componentes interactúa de manera coordinada para ofrecer al usuario una experiencia fluida y cohesiva.

A nivel de interfaz gráfica, Android proporciona una jerarquía robusta de elementos visuales conocida como View y ViewGroups. Las Views son los bloques básicos de construcción de la UI —por ejemplo, botones, campos de texto, imágenes—, mientras que los ViewGroups son contenedores que organizan y agrupan múltiples Views, como los LinearLayout, RelativeLayout o ConstraintLayout. El dominio de estos conceptos permite al desarrollador diseñar interfaces eficientes y adaptables, lo que es especialmente relevante en un entorno tan diverso como el ecosistema Android, donde los dispositivos pueden variar ampliamente en tamaño, resolución y orientación.

Por tanto, esta sesión se enfoca en que los estudiantes comprendan no solo los roles de los principales componentes de una aplicación Android, sino también cómo organizar la interfaz gráfica utilizando Views y ViewGroups para lograr aplicaciones más intuitivas, responsivas y profesionales.

I. COMPONENTES DE UNA APLICACIÓN ANDROID

Una aplicación Android está construida sobre una arquitectura modular que se compone de varios bloques de construcción fundamentales, cada uno con un propósito específico. Estos componentes pueden interactuar entre sí para lograr la funcionalidad deseada por el usuario. Los principales componentes de una aplicación Android son:

1) Actividades (Activities)

a) Concepto: Una Activity representa una única pantalla con una interfaz de usuario. Es el punto de interacción principal entre el usuario y la aplicación para una tarea específica. Una aplicación puede contener múltiples actividades, y el usuario navega entre ellas para realizar diferentes acciones.

b) Características:

- **Ciclo de Vida:** Las Activities tienen un ciclo de vida bien definido (creación, inicio, reanudación, pausa, detención, destrucción) que el sistema operativo Android gestiona. Los desarrolladores deben comprender y manejar estos estados para asegurar el comportamiento correcto de la aplicación.
- **Interfaz de Usuario:** Cada Activity está asociada a una interfaz de usuario (layout) definida típicamente en XML, que contiene las Views y los ViewGroups que se muestran al usuario.
- **Interacción del Usuario:** Las Activities son responsables de manejar los eventos de entrada del usuario (toques, clics, etc.) y actualizar la interfaz de usuario en consecuencia.

- **Stack de Actividades (Back Stack):** Android gestiona una pila de actividades visitadas por el usuario dentro de una aplicación. Al presionar el botón "Atrás", la actividad superior de la pila se finaliza y la actividad anterior se vuelve a mostrar.

c) Ejemplos:

- En una aplicación de correo electrónico, una Activity podría mostrar la lista de correos, otra podría permitir redactar un nuevo correo, y una tercera podría mostrar el contenido de un correo específico.
- En una aplicación de redes sociales, una Activity podría mostrar el feed de noticias, otra el perfil del usuario y otra la ventana para publicar un nuevo estado.

d) Código de Ejemplo (Kotlin - Fragmento de una Activity):

```
class MainActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main) // Asocia el layout XML a  
esta Activity  
  
        val miBoton: Button = findViewById(R.id.miBoton)  
        miBoton.setOnClickListener {  
            // Lógica a ejecutar cuando se hace clic en el botón  
            val intent = Intent(this, SecondActivity::class.java)  
            startActivity(intent) // Inicia una nueva Activity  
        }  
    }  
}
```

```
override fun onStart() {  
    super.onStart()  
    // Código a ejecutar cuando la Activity se vuelve visible  
}
```

```
override fun onResume() {  
    super.onResume()  
    // Código a ejecutar cuando la Activity comienza a interactuar con  
    el usuario  
}  
  
// ... otros métodos del ciclo de vida  
}
```

1.1. Fragmentos (Fragments):

- **Concepto:** Un Fragment representa una porción reutilizable de la interfaz de usuario de una Activity. Un Activity puede contener múltiples Fragments en la pantalla simultáneamente. Los Fragments tienen su propio ciclo de vida, manejan sus propios eventos de entrada y pueden ser añadidos, reemplazados o removidos de una Activity en tiempo de ejecución.
- **Características:**
 - **Reutilización de UI:** Permiten reutilizar partes de la interfaz de usuario en diferentes Activities.
 - **Adaptación a Diferentes Tamaños de Pantalla:** Facilitan la creación de interfaces adaptativas que se ven bien tanto en teléfonos como en tablets (ej., mostrando una lista y los detalles en la misma pantalla en tablets).

- **Ciclo de Vida Propio:** Los Fragments tienen su propio ciclo de vida que está ligado al ciclo de vida de la Activity que los contiene.
- **Gestión por el FragmentManager:** Los Fragments son gestionados por la FragmentManager dentro de una Activity.
- **Ejemplos:**
 - a) En una aplicación de noticias en una tablet, una Activity podría mostrar una lista de artículos en un Fragment a la izquierda y el contenido del artículo seleccionado en otro Fragment a la derecha. En un teléfono, estos Fragments podrían mostrarse en Activities separadas.
 - b) Una interfaz de usuario con pestañas podría implementar cada pestaña como un Fragment.

- **Código de Ejemplo (Kotlin - Fragmento):**

```
class MiFragmento : Fragment() {

    override fun onCreateView(inflater: LayoutInflater, container:
    ViewGroup?, savedInstanceState: Bundle?): View? {
        // Infla el layout para este fragmento
        return inflater.inflate(R.layout.fragment_mi, container,
        false)
    }

    override fun onViewCreated(view: View, savedInstanceState:
    Bundle?) {
        super.onViewCreated(view, savedInstanceState)
    }
}
```

```
// Código a ejecutar después de que la vista del fragmento
ha sido creada

val miTextView: TextView =
view.findViewById(R.id.miTextViewEnFragmento)
miTextView.text = "Texto desde el Fragmento"
}

// ... otros métodos del ciclo de vida del Fragmento
}
```

2) Servicios (Services)

a) Concepto: Un Service es un componente que se ejecuta en segundo plano para realizar operaciones que no requieren una interfaz de usuario visible o que deben continuar ejecutándose incluso si el usuario interactúa con otra aplicación.

b) Características:

- **Ejecución en Segundo Plano:** Los Servicios se ejecutan en el hilo principal de la aplicación por defecto, por lo que las operaciones largas deben realizarse en hilos separados para evitar bloquear la interfaz de usuarios.
- **Sin Interfaz de Usuario Directa:** Los Servicios no proporcionan una interfaz visual directa al usuario. Pueden mostrar notificaciones para informar al usuario sobre su estado.
- **Ciclo de Vida:** Los Servicios también tienen un ciclo de vida (creación, inicio, ejecución, destrucción) gestionado por el sistema.
- **Diferentes Tipos:** Existen principalmente dos tipos de Servicios.
 - **Started Services:** Se inician mediante una llamada a `startService()` y generalmente realizan una tarea específica sin

devolver un resultado al iniciador. Continúan ejecutándose hasta que terminan su tarea o son detenidos explícitamente.

- **Bound Services:** Se enlazan a otros componentes (generalmente Activities) mediante una llamada a `bindService()`. Proporcionan una interfaz para que los componentes enlazados interactúen con el servicio (ej., enviar comandos, recibir resultados). El servicio se ejecuta mientras haya componentes enlazados a él.

c) Ejemplos:

- Una aplicación de reproducción de música utiliza un Servicio para reproducir música en segundo plano, incluso cuando el usuario cambia a otra aplicación o la pantalla se apaga.
- Una aplicación de descarga de archivos utiliza un Servicio para descargar archivos de la red sin bloquear la interfaz de usuario.
- Un Servicio puede proporcionar datos a una Activity a través del enlace (Bound Service).

d) Código de Ejemplo (Kotlin - Fragmento de un Servicio Started)

```
class MiServicio : Service() {

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int
    {
        // Código a ejecutar cuando se inicia el servicio
        doLongRunningTask()
        return START_STICKY // Indica al sistema que reinicie el servicio si
        se detiene inesperadamente
    }
}
```

```
override fun onBind(intent: Intent?): IBinder? {  
    // Se llama cuando otro componente quiere enlazarse al servicio  
    (para Bound Services)  
    return null  
}
```

```
private fun doLongRunningTask() {  
    // Simulación de una tarea larga en un hilo separado  
    Thread {  
        for (i in 1..10) {  
            Log.d("MiServicio", "Tarea en progreso: $i")  
            Thread.sleep(1000)  
        }  
        stopSelf() // Detiene el servicio cuando la tarea ha terminado  
    }.start()  
}
```

```
override fun onDestroy() {  
    super.onDestroy()  
    Log.d("MiServicio", "Servicio destruido")  
}  
}
```

3) Proveedores de Contenido (Content Providers)

a) Concepto: Un Content Provider gestiona el acceso a un conjunto estructurado de datos. Actúa como una interfaz estándar para que otras aplicaciones puedan consultar y modificar estos datos (si tienen los permisos adecuados), abstrayendo los detalles de la implementación subyacente (ej., cómo se almacenan los datos, si es en una base de datos SQLite, archivos, o en la red).

b) Características y Funciones Clave:

- **Abstracción de Datos:** Ocultan los detalles de almacenamiento de los datos.
- **Acceso Controlado:** Proporcionan mecanismos para definir permisos y controlar qué aplicaciones pueden acceder a los datos y cómo.
- **Interfaz Estándar:** Implementan un conjunto estándar de métodos de consulta (`query()`), inserción (`insert()`), actualización (`update()`) y eliminación (`delete()`) que las otras aplicaciones pueden utilizar.
- **URLs de Contenido:** Los datos expuestos por un Content Provider se identifican mediante URLs de contenido (Uniform Resource Identifiers).

c) Ejemplos:

- El sistema Android proporciona Content Providers para acceder a los contactos (`ContactsContract`), la biblioteca multimedia (`MediaStore`), los calendarios (`CalendarContract`), etc.
- Una aplicación podría crear su propio Content Provider para compartir sus datos con otras aplicaciones (ej., una aplicación de notas podría permitir que otras aplicaciones lean las notas).

d) Código de Ejemplo (Kotlin - Fragmento de un Content Provider)

```
class MiProveedorDeContenido : ContentProvider() {

    private lateinit var dbHelper: MiBaseDeDatosHelper

    private val uriMatcher = UriMatcher(UriMatcher.NO_MATCH).apply {
        addURI(AUTHORITY, "notas", NOTAS)
        addURI(AUTHORITY, "notas/#", NOTA_ID)
    }

    companion object {
        const val AUTHORITY = "com.example.minota.provider"
        private const val NOTAS = 1
        private const val NOTA_ID = 2
    }

    override fun onCreate(): Boolean {
        dbHelper = MiBaseDeDatosHelper(context)
        return true
    }

    override fun query(uri: Uri, projection: Array<out String>?, selection:
String?, selectionArgs: Array<out String>?, sortOrder: String?): Cursor? {
        val db = dbHelper.readableDatabase
        val cursor: Cursor? = when (uriMatcher.match(uri)) {
            NOTAS -> db.query("notas", projection, selection, selectionArgs,
null, null, sortOrder)
```

```

        NOTA_ID -> db.query("notas", projection, "_id = ?",
        arrayOf(uri.lastPathSegment), null, null, sortOrder)

        else -> throw IllegalArgumentException("URI desconocido: $uri")
    }

    cursor?.setNotificationUri(context?.contentResolver, uri)

    return cursor
}

// ... (implementaciones para insert, update, delete, getType)
}

```

4) Receptores de Difusión (Broadcast Receivers)

a) Concepto: Un Broadcast Receiver es un componente que responde a anuncios de difusión (broadcast intents) enviados por el sistema operativo o por otras aplicaciones. Los intents de difusión son un mecanismo para la comunicación entre aplicaciones y el sistema, o entre aplicaciones.

b) Características:

- **Respuesta a Eventos:** Los Broadcast Receivers se registran para escuchar intents específicos. Cuando se emite un intent que coincide con el filtro del receptor, se ejecuta el método `onReceive()` del receptor.
- **Duración Corta:** Los Broadcast Receivers están diseñados para realizar tareas pequeñas y rápidas en respuesta a un intent de difusión. Para tareas más largas, deben delegar el trabajo a un Servicio.

- **Registro:** Los receptores pueden registrarse dinámicamente en el código (con un `IntentFilter`) o declararse estáticamente en el archivo `AndroidManifest.xml`.

c) Ejemplos:

- Una aplicación puede registrar un `Broadcast Receiver` para recibir notificaciones sobre cambios en el estado de la batería (bajo nivel de batería), la conectividad de red (cambio de Wi-Fi a datos móviles), o el arranque del dispositivo.
- El sistema operativo emite broadcasts cuando se instala o desinstala una aplicación.
- Las aplicaciones también pueden enviar sus propios broadcasts para comunicar eventos a otras aplicaciones.

d) Código de Ejemplo (Kotlin - Fragmento de un Broadcast Receiver)

```
class MiReceptorDeDifusion : BroadcastReceiver() {

    override fun onReceive(context: Context?, intent: Intent?) {
        if (intent?.action == Intent.ACTION_BATTERY_LOW) {
            // Realizar acciones cuando la batería está baja
            Log.w("MiReceptor", "¡Batería baja!")
            context?.let {
                Toast.makeText(it, "Batería baja", Toast.LENGTH_LONG).show()
            }
        } else if (intent?.action == Intent.ACTION_BOOT_COMPLETED) {
            // Realizar acciones después de que el dispositivo se ha iniciado
            Log.i("MiReceptor", "Dispositivo iniciado")
            // Iniciar un servicio al arrancar, por ejemplo
        }
    }
}
```

```

        context?.startService(Intent(context,
        MiServicioDeInicio::class.java))
    }
}
}

```

Para registrar este receptor estáticamente en el AndroidManifest.xml:

```

<receiver android:name=".MiReceptorDeDifusion"
    android:enabled="true"
    android:exported="false">
    <intent-filter>
        <action android:name="android.intent.action.BATTERY_LOW" />
    </intent-filter>
    <intent-filter>
        <action    android:name="android.intent.action.BOOT_COMPLETED"
    />
        <category android:name="android.intent.category.DEFAULT" />
    </intent-filter>
</receiver>

```

5) Framework de Aplicaciones (Application Framework)

a) Concepto: El framework de aplicaciones proporciona un conjunto de APIs de alto nivel y bloques de construcción que los desarrolladores utilizan para crear aplicaciones de Android. Simplifica el proceso de desarrollo al proporcionar abstracciones y servicios comunes.

b) Características y Funciones Clave:

- **Componentes de la Aplicación:** Define la estructura básica de las aplicaciones de Android a través de componentes como Activities

(interfaces de usuario), Services (procesos en segundo plano), Content Providers (gestión de datos compartidos) y Broadcast Receivers (receptores de eventos del sistema).

- **Gestión de la Interfaz de Usuario:** Proporciona herramientas y clases para crear interfaces de usuario interactivas (Views, Layouts, Widgets).
- **Administración de Recursos:** Permite acceder a recursos de la aplicación como imágenes, archivos de diseño, cadenas de texto, etc.
- **Administración de Paquetes:** Gestiona la instalación, desinstalación y la información de las aplicaciones.
- **Administración de la Telefonía:** Proporciona APIs para acceder a las funcionalidades de telefonía (llamadas, SMS).
- **Administración de la Ubicación:** Permite acceder a la información de ubicación del dispositivo.
- **Notificaciones:** Proporciona mecanismos para mostrar notificaciones al usuario.
- **Intents:** Un mecanismo para la comunicación asíncrona entre componentes de la aplicación y otras aplicaciones.

c) Ejemplos: Cuando un desarrollador crea una interfaz de usuario con botones y campos de texto, está utilizando las clases y APIs proporcionadas por el framework de aplicaciones (Views, TextView, Button, LinearLayout, etc.). Cuando una aplicación quiere enviar una notificación, utiliza las APIs del NotificationManager del framework.

6) Aplicaciones (Applications)

a) Concepto: Esta es la capa superior de la arquitectura de Android y contiene todas las aplicaciones instaladas en el dispositivo, tanto las aplicaciones del sistema (preinstaladas) como las aplicaciones de terceros instaladas por el usuario.

b) Características:

- **Desarrolladas utilizando el Framework de Aplicaciones:** Todas las aplicaciones se construyen utilizando las APIs y los componentes proporcionados por el framework de aplicaciones.
- **Diversidad de Funcionalidades:** Cubren una amplia gama de funcionalidades, desde herramientas de productividad y comunicación hasta juegos y aplicaciones de entretenimiento.
- **Ejecutadas en el ART:** Se ejecutan dentro del entorno ART.
- **Interactúan con las Capas Inferiores:** Utilizan las APIs del framework de aplicaciones para acceder a los servicios y recursos proporcionados por las capas inferiores (acceso al hardware a través del HAL, almacenamiento a través del kernel, etc.).

c) Ejemplos: Gmail, Chrome, WhatsApp, juegos, aplicaciones de redes sociales, etc.

Ejemplo Práctico

Cuando un usuario abre la aplicación de cámara:

- 1) La aplicación solicita acceso a la cámara a través del framework de aplicaciones.
- 2) El framework utiliza las bibliotecas nativas para comunicarse con el hardware.
- 3) Las solicitudes se transmiten al núcleo de Linux, que interactúa con los controladores de hardware para activar la cámara.
- 4) La imagen capturada se procesa y se muestra en la interfaz de usuario.

Este flujo demuestra cómo las diferentes capas de la arquitectura de Android trabajan conjuntamente para proporcionar funcionalidades a las aplicaciones

II. VIEWS Y VIEWGROUPS

Una aplicación Android está compuesta por varios componentes esenciales que trabajan juntos para proporcionar la funcionalidad deseada al usuario. Entre estos, las Views y los ViewGroups son fundamentales para la creación de la interfaz de usuario (UI).

1) VIEWS

a) Concepto: Una View es el bloque de construcción fundamental para la interfaz de usuario en Android. Representa un área rectangular en la pantalla y es responsable de dibujar algo y manejar la interacción del usuario. Casi todos los elementos visuales que el usuario ve e interactúa en una aplicación Android son instancias de la clase View o de sus subclases.

b) Características:

- **Dibujo:** Las Views tienen la capacidad de dibujarse a sí mismas en la pantalla. Definen cómo se ven y cómo responden a los cambios.
- **Manejo de Eventos:** Las Views pueden detectar y manejar eventos de entrada del usuario, como clics, toques, deslizamientos y eventos del sistema.
- **Atributos:** Las Views tienen una gran cantidad de atributos que definen su apariencia (color de fondo, texto, tamaño, visibilidad, etc.) y su comportamiento (manejadores de clics, capacidad de

desplazamiento, etc.). Estos atributos se pueden configurar en el archivo de diseño XML o dinámicamente en el código Java/Kotlin.

- **Clase Base:** La clase `android.view.View` es la clase base para todos los componentes visuales.

c) Ejemplos de Subclases Comunes de View:

- **TextView:** Muestra texto en la pantalla.
- **EditText:** Permite al usuario ingresar y editar texto.
- **Button:** Un botón interactivo que realiza una acción cuando se hace clic.
- **ImageView:** Muestra imágenes.
- **CheckBox:** Una casilla de verificación que permite al usuario seleccionar o deseleccionar una opción.
- **RadioButton:** Un botón de opción que pertenece a un grupo, donde solo una opción puede estar seleccionada.
- **Switch:** Un interruptor de encendido/apagado.
- **ProgressBar:** Muestra el progreso de una tarea.
- **SeekBar:** Permite al usuario seleccionar un valor dentro de un rango.

d) Ejemplo de Uso en XML:

```
<TextView
    android:id="@+id/textViewSaludo"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="¡Hola, mundo!"
    android:textSize="24sp"
    android:textColor="@android:color/black" />
```

```
<Button  
    android:id="@+id/botonAccion"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="Haz clic aquí" />
```

```
<ImageView  
    android:id="@+id/imageViewLogo"  
    android:layout_width="100dp"  
    android:layout_height="100dp"  
    android:src="@drawable/logo" />
```

e) Ejemplo de Manipulación en Código (Kotlin):

```
val saludoTextView: TextView = findViewById(R.id.textViewSaludo)  
val accionButton: Button = findViewById(R.id.botonAccion)  
  
accionButton.setOnClickListener {  
    saludoTextView.text = "¡Botón pulsado!"  
}
```

2) ViewGroups

a) Concepto: Un ViewGroup es una subclase de View que puede contener otras Views (incluyendo otros ViewGroups) y define la estructura de diseño de la interfaz de usuario. Los ViewGroups son contenedores invisibles que organizan y controlan la disposición de sus Views hijas en la pantalla. Definen las reglas de diseño (layout) para sus hijos.

b) Características:

- **Contenedores:** Actúan como contenedores para otras Views.

- **Gestión del Layout:** Implementan diferentes algoritmos de diseño para determinar cómo se posicionan y dimensionan sus Views hijas.
- **Pueden ser Anidados:** Los ViewGroups pueden contener otros ViewGroups, lo que permite crear jerarquías de diseño complejas.
- **Clase Base:** La clase `android.view.ViewGroup` es la clase base para todos los layouts.

c) Ejemplos de Subclases Comunes de ViewGroup (Layouts):

- **CoordinatorLayout:** es un ViewGroup potente y versátil en Android, diseñado para coordinar las interacciones entre sus vistas secundarias. Se introdujo con la Design Support Library (ahora parte de AndroidX) y es fundamental para implementar patrones de diseño complejos y animaciones sincronizadas, especialmente aquellos relacionados con el desplazamiento (scrolling)
- **ConstraintLayout:** es un ViewGroup potente y flexible en Android diseñado para crear interfaces de usuario complejas con una estructura plana (sin anidamiento excesivo de layouts). Se introdujo en Android Studio 2.2. y se ha convertido en el layout recomendado por Google para la mayoría de los diseños modernos debido a su eficiencia y versatilidad.
- **LinearLayout:** Organiza las Views hijas en una sola dirección, ya sea horizontal o verticalmente.
- **RelativeLayout:** Permite posicionar las Views hijas relativas entre sí o relativas al ViewGroup padre.
- **FrameLayout:** Un contenedor simple que apila las Views hijas una encima de la otra. La última View agregada se dibuja en la parte superior.

- **ScrollView:** Permite desplazar el contenido si es más grande que la pantalla. Puede ser horizontal (HorizontalScrollView) o vertical. Solo puede contener un único View hijo directo.
- **RecyclerView:** Un ViewGroup avanzado y eficiente para mostrar grandes conjuntos de datos en listas o cuadrículas. Proporciona mecanismos para la reutilización de Views (ViewHolders) para mejorar el rendimiento.
- **GridLayout:** Organiza las Views hijas en una cuadrícula.

d) Ejemplo de Uso en XML (LinearLayout Vertical):

```
<LinearLayout
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:orientation="vertical">
```

```
    <TextView
```

```
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
```

```
        android:text="Título principal"
```

```
        android:textSize="20sp" />
```

```
    <EditText
```

```
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
```

```
        android:hint="Ingrese su nombre" />
```

```
    <Button
```

```
        android:layout_width="wrap_content"
```

```
android:layout_height="wrap_content"
android:text="Enviar" />
```

```
</LinearLayout>
```

e) Ejemplo de Uso en XML (ConstraintLayout):

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
```

```
<TextView
    android:id="@+id/textViewTitulo"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Título"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

```
<Button
    android:id="@+id/botonAceptar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Aceptar"
    app:layout_constraintTop_toBottomOf="@+id/textViewTitulo"
    app:layout_constraintStart_toStartOf="parent" />
```

```
<Button  
    android:id="@+id/botonCancelar"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="Cancelar"  
    app:layout_constraintTop_toBottomOf="@+id/textViewTitulo"  
    app:layout_constraintEnd_toEndOf="parent" />  
  
</androidx.constraintlayout.widget.ConstraintLayout>
```

3) ConstraintLayout vs CoordinatorLayout

En el desarrollo de interfaces de usuario en Android, tanto ConstraintLayout como CoordinatorLayout son ViewGroups poderosos, pero están diseñados para propósitos distintos.

A. ConstraintLayout:

- **Propósito Principal:** Diseñado para crear layouts complejos con una jerarquía plana. Su objetivo es evitar el anidamiento profundo de ViewGroups como LinearLayout o RelativeLayout, lo que puede impactar negativamente en el rendimiento del renderizado de la interfaz de usuario.
- **Mecanismo Clave:** Utiliza un sistema de restricciones (constraints) para definir las relaciones de posición y tamaño entre las vistas hijas. Estas restricciones pueden ser relativas al padre, a otras vistas hermanas o a guías virtuales.
- **Casos de Uso Típicos:**

- Creación de diseños de pantalla individuales que requieren un posicionamiento flexible y adaptabilidad a diferentes tamaños de pantalla.
 - Implementación de interfaces de usuario complejas sin anidar múltiples layouts.
 - Construcción de filas o celdas personalizadas en RecyclerView o ListView.
 - Servir como el layout base para fragmentos o actividades donde se necesita un control preciso sobre la ubicación de los elementos.
 - Fundamento para animaciones complejas utilizando MotionLayout.
- **Comportamiento:** Se centra en cómo se posicionan y dimensionan las vistas dentro de sus límites, basándose en las restricciones definidas. No tiene un comportamiento inherente para coordinar interacciones entre vistas de forma dinámica en respuesta a eventos.

B. CoordinatorLayout:

- **Propósito Principal:** Diseñado para coordinar las interacciones entre sus vistas secundarias. Es un FrameLayout con esteroides que proporciona un mecanismo para que las vistas hijas reaccionen a los eventos que ocurren dentro del layout o en otras vistas hijas.
- **Mecanismo Clave:** Utiliza un concepto llamado Coordinator.Behavior. Un Behavior se adjunta a una vista hija y permite interceptar eventos (como toques, deslizamientos, cambios de tamaño) y modificar el comportamiento de esa vista en respuesta a esos eventos o a cambios en otras vistas.
- **Casos de Uso Típicos:**

- Implementación de patrones de desplazamiento complejos, como la desaparición de la Toolbar al hacer scroll en un RecyclerView (AppBarLayout).
 - Coordinación de animaciones sincronizadas entre múltiples vistas.
 - Definición de dependencias entre vistas, como el movimiento de un FloatingActionButton al aparecer una Snackbar.
 - Actuar como el layout raíz para actividades o fragmentos que implementan interacciones coordinadas, especialmente con componentes de Material Design como AppBarLayout, FloatingActionButton, Snackbar, etc.
 - Servir como el contenedor principal para la estructura de la interfaz de usuario de una aplicación, permitiendo la coordinación a nivel de pantalla.
- **Comportamiento:** Se centra en cómo las vistas interactúan y responden dinámicamente entre sí en función de eventos y dependencias definidas a través de los Behaviors.

4) Relación entre Views y ViewGroups

La interfaz de usuario de una aplicación Android se construye típicamente como una jerarquía de Views y ViewGroups. El ViewGroup raíz (a menudo un ConstraintLayout, LinearLayout o FrameLayout) contiene otras Views y/o ViewGroups, que a su vez pueden contener más Views y ViewGroups, formando un árbol de la interfaz de usuario. El sistema Android recorre este árbol para dibujar la interfaz y manejar los eventos.